```
from tensorflow.keras.datasets import fashion_mnist
import matplotlib.pyplot as plt
import numpy as np


(fashion_x, _), _ = fashion_mnist.load_data()
```

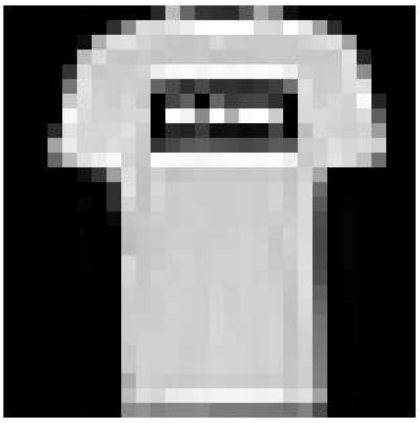
Task 1

```
image = fashion_x[1]
image.shape
```

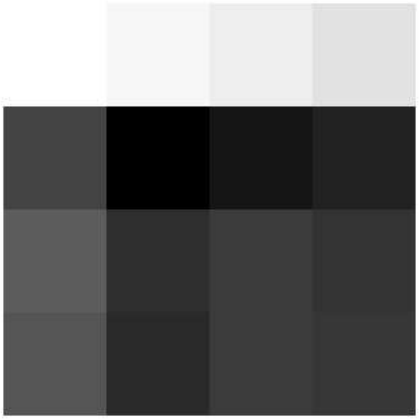
```
(28, 28)
```

```
def show_img(img):
  plt.figure(figsize=(6, 4))
  plt.imshow(img, cmap='grey')
  plt.axis('off')
  plt.show()
```

```
show_img(image)
```



```
patch = image[10:14, 10:14]
show_img(patch)
```



```
def max_pooling(image, pool_size=2, stride=2):

  (img_h, img_w) = image.shape

  out_h, out_w = ((img_h - pool_size) // stride) + 1, ((img_w - pool_size) // stride) + 1

  output = np.zeros((out_h, out_w))
```

```
  for r in range(out_h):
    for c in range(out_w):
        y = r * stride
        x = c * stride

        region = image[y: y + pool_size, x: x + pool_size]
        output[r, c] = np.max(region)

  return output
```

```
def avg_pooling(image, pool_size=2, stride=2):

  (img_h, img_w) = image.shape

  out_h, out_w = ((img_h - pool_size) // stride) + 1, ((img_w - pool_size) // stride) + 1

  output = np.zeros((out_h, out_w))

  for r in range(out_h):
    for c in range(out_w):
        y = r * stride
        x = c * stride

        region = image[y: y + pool_size, x: x + pool_size]
        output[r, c] = np.mean(region)

  return output
```

```
max_pooled = max_pooling(patch)

avg_pooled = avg_pooling(patch)


show_img(max_pooled)
show_img(avg_pooled)
```